Übungsblatt 3: Threads, Präemption und zeitgesteuertes Warten

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziel und Voraussetzungen

Sie übertragen das Aufgabenmodell aus Vorlesung 2 auf einen präemptiven FreeRTOS-Kernel. Sie unterscheiden bereit, laufend und wartend, begründen eine Prozessorbelegung und planen Speicher sowie periodische Freigaben. Anschließend beobachten Sie dieselben Mechanismen auf einem **NUCLEO-H755ZI-Q** mit **STM32H755ZIT6**.

RTOS bedeutet Real-Time Operating System, Echtzeitbetriebssystem. Ein Thread ist ein fortsetzbarer Ausführungsablauf; FreeRTOS nennt ihn Task. Ein Job ist eine einzelne Aktivierung. CMSIS-RTOS2 ist die Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard. Im Programmierteil läuft genau ein Kernel auf dem Cortex-M4. Der Cortex-M7 behält die System- und Startfunktion aus Vorlesung 1.

Teil A ist ohne Hardware vollständig lösbar und klausurrelevant. Bearbeiten Sie ihn vor Teil B. Teil B demonstriert die Auswahlregel durch Programmieren mit **STM32CubeMX und VS Code mit STM32 VS Code Extension**. Ein Oszilloskop ist nur für die zusätzliche Messaufgabe erforderlich. Im Repository liegt kein vollständiges Firmwareprojekt; der C-Baustein unter `code/lab03/` ist Anwendungscode zur Integration in ein erzeugtes Projekt.

Teil A: Papieraufgaben

Aufgabe 1: Vom Callback zum Thread (6 Punkte)

1. Erklären Sie den Unterschied zwischen der endlichen `step()`-Funktion des Dispatchers aus Vorlesung 2 und der Eintrittsfunktion eines RTOS-Threads. Warum ist eine Endlosschleife im ersten Fall problematisch, im zweiten nicht grundsätzlich?
2. Ein Thread ruft `osDelay(5)` auf. Beschreiben Sie die Zustandsfolge bis zur erneuten Ausführung. Bedeutet das Ablaufen der Zeit automatisch, dass er läuft?
3. Heartbeat ruft stattdessen die übliche `HAL_Delay(...)` auf. Was verändert sich für die Prozessorzuteilung? Sind Interrupts dadurch automatisch gesperrt?

Notieren Sie Ihre Begründungen, nicht nur die Zustandsnamen.

Lösung und Bewertung: Je zwei Punkte pro begründeter Teilantwort.

1. Der Dispatcher erhält die Kontrolle erst nach Rückkehr aus `step()`; dessen Fortschritt wird ausdrücklich in Anwendungsdaten gespeichert. Ein nicht zurückkehrender Schritt verhindert weitere Callback-Aufrufe. Interrupts können trotzdem laufen. Ein RTOS-Thread besitzt dagegen einen gesicherten Fortsetzungszustand und eigenen Stack. Er kann durch einen Warteaufruf blockieren oder von einer höheren bereiten Task verdrängt werden. Eine hochpriore, dauerhaft bereite Endlosschleife bleibt trotzdem problematisch.
2. Running → Blocked beim Warteaufruf, Blocked → Ready nach Ablauf der fünf **Kernelticks**, Ready → Running erst nach Zuteilung. Eine höhere bereite Priorität kann den Start weiter verzögern.
3. `HAL_Delay` ist in der üblichen Hardware Abstraction Layer (HAL) aktives Warten. Heartbeat bleibt ausführungsbereit und verbraucht seine zugeteilte Zeit. Er wird dadurch nicht durch die Kernel-Warteverwaltung Blocked. Höhere Taskprioritäten können ihn weiterhin präemptieren, und erlaubte Interrupts können laufen.

Aufgabe 2: Zustände, Präemption und Fristen (10 Punkte)

Didaktisches **Ein-Kern-Modell**, keine Boardmessung:

Aufgabe	Priorität	Freigaben / Verhalten	CPU-Zeit pro Aktivierung
Ereignis-Task	höchste	wartet zunächst; Ereignis bei 2 ms	1 ms, danach erneut warten
Heartbeat	mittlere	periodisch bei 0, 4, 8 ms	1 ms, danach bis zur nächsten Freigabe warten
HeavyMath	niedrigste Anwenderpriorität	immer bereit	unbegrenzte Hintergrundarbeit

CPU bedeutet Central Processing Unit, zentrale Verarbeitungseinheit. Alle Wechsel-, Interrupt- und Warteverwaltungszeiten sind hier null. Der Ereignis-Thread ist bei $t = 0$ bereits wartend. Für Heartbeat gilt $D = 4\,\text{ms}$ als relative Deadline.

1. Zeichnen Sie die CPU-Belegung im halboffenen Intervall $[0, 10)$ ms. Kennzeichnen Sie jeden Wechsel mit seiner Ursache. (4 Punkte)
2. Geben Sie die drei Taskzustände bei 1,5 und 2,5 ms an. (2 Punkte)
3. Berechnen Sie die insgesamt erhaltene CPU-Zeit jeder Aufgabe und die drei Heartbeat-Antwortzeiten. (2 Punkte)
4. Zusätzlich wird die Ereignis-Task bei 4 ms erneut bereit. Wann startet und endet der Heartbeat-Job dieser Freigabe, und hält er seine Deadline? (2 Punkte)

CPU-Belegung:

Intervall in ms	CPU	Ursache am Anfang
$[0, 1)$	Heartbeat	höchste bereite Priorität
$[1, 2)$	HeavyMath	Heartbeat blockiert
$[2, 3)$	Ereignis-Task	Ereignis macht höchste Priorität bereit
$[3, 4)$	HeavyMath	Ereignis-Task blockiert wieder
$[4, 5)$	Heartbeat	periodische Freigabe
$[5, 8)$	HeavyMath	Heartbeat blockiert
$[8, 9)$	Heartbeat	periodische Freigabe
$[9, 10)$	HeavyMath	Heartbeat blockiert

Intervall in ms	CPU	Ursache am Anfang

Bei 1,5 ms: Ereignis-Task Blocked, Heartbeat Blocked, HeavyMath Running. Bei 2,5 ms: Ereignis-Task Running, Heartbeat Blocked, HeavyMath Ready. Die verdrängte HeavyMath-Task wartet auf Zuteilung, nicht auf eine sachliche Wartebedingung.

Im ursprünglichen Modell erhält Heartbeat 3 ms, die Ereignis-Task 1 ms und HeavyMath 6 ms. Die Antwortzeiten $R = f - r$ sind jeweils 1 ms: Fertigstellung f minus Freigabe r.

Mit der zusätzlichen Ereignisfreigabe läuft die Ereignis-Task von 4 bis 5, Heartbeat von 5 bis 6. Somit $R = 6 - 4 = 2\,\text{ms} \leq D = 4\,\text{ms}$. Der Start ist verzögert, die Modellfrist trotzdem eingehalten. Bei $D = 1\,\text{ms}$ wäre sie verletzt. Diese Rechnung gilt nur unter den genannten Modellannahmen.

Aufgabe 3: Ticks, Drift und ein verpasstes Ziel (8 Punkte)

1. Ein Kernel läuft mit $f_{\text{tick}} = 100\,\text{Hz}$. Bestimmen Sie Tickperiode und aufgerundete Tickzahl für 5 ms und 500 ms. Warum bedeutet `osDelay(1)` nicht eine garantierte Mindestwartezeit von 10 ms? (3 Punkte)
2. Anderes Modell: Tickgrenzen in 1-ms-Abständen, Arbeitsdauern nacheinander 2, 4 und 1 ms; der erste Start liegt bei null. Alle Arbeitsenden liegen genau auf Tickgrenzen, weitere Störungen fehlen. Berechnen Sie die nächsten drei Starts bei „Arbeit, dann `osDelay(10)`“ und bei einem absoluten 10-Tick-Sollraster. (3 Punkte)
3. Ein Sollziel liegt bei Tick 20, die Arbeit endet erst bei Tick 23. Nennen Sie eine sinnvolle Reaktion mit Periode 10 Ticks und erklären Sie, warum nicht sofort mehrere LED-Umschaltungen nachgeholt werden sollten. (2 Punkte)

1. $T_{\text{tick}} = 1/100\,\text{s} = 10\,\text{ms}$. Für t Millisekunden gilt $n = \lceil t f_{\text{tick}}/1000 \rceil$. Fünf Millisekunden ergeben $\lceil 0{,}5 \rceil = 1$ Tick, 500 Millisekunden 50 Ticks. Erfolgt `osDelay(1)` eine Millisekunde vor der nächsten Tickgrenze, wird der Thread schon nach ungefähr einer Millisekunde Ready. Die Tickphase bestimmt den Rest; höhere Last kann den anschließenden Start wieder verzögern.
2. Relative Starts: $0 + 2 + 10 = 12$, $12 + 4 + 10 = 26$, $26 + 1 + 10 = 37\,\text{ms}$. Absolute Starts im Modell: 10, 20, 30 ms. Die Arbeit passt jeweils vor den nächsten Sollpunkt. Das absolute Raster addiert die Arbeitsdauer nicht zur Periode. Es garantiert unter Störungen nicht den tatsächlichen Start.
3. Laborpolitik: Zielverfehlung zählen, neues Ziel $23 + 10 = 33$ setzen und bis dahin blockieren. Ein direktes mehrfaches Nachholen würde Ereignisse künstlich zusammenballen und die Last verschärfen. Alternativ kann ein spezifiziertes System veraltete Aktivierungen überspringen oder einen Fehler melden. Der Warteziel-Zähler ist ohne zusätzliche Deadlinedefinition kein allgemeiner Deadline-Miss-Zähler.

Aufgabe 4: Speicher, Prioritäten und Ereignisse (8 Punkte)

1. CMSIS-Stacks 512, 1024 und 1024 Byte; angenommener Verwaltungsblock (TCB, Task Control Block) jeweils 80 Byte. Berechnen Sie den Bedarf dieser drei Stack-/TCB-Paare. Warum reicht diese Rechnung noch nicht als Heapnachweis? Wie viele Byte sind 128 native `StackType_t`-Elemente zu je vier Byte? (3 Punkte)

2. HeavyMath wird über Heartbeat priorisiert und bleibt dauerhaft Ready. Heartbeat ist ebenfalls Ready. Hilft `osThreadYield()` in HeavyMath? Ist das Priority Inversion? (3 Punkte)
3. Die logische Interruptschwelle für Kernelaufrufe sei 5, mit vier implementierten Prioritätsbits und passender Gruppierung. Darf ein IRQ mit logischer Priorität 3 `osThreadFlagsSet` verwenden? Darf ein erlaubter IRQ `osThreadFlagsWait` verwenden? Zählt zweimaliges Setzen desselben Bits vor der Verarbeitung zwei Ereignisse? (2 Punkte)

1. $512 + 1024 + 1024 + 3 \cdot 80 = 2800$ Byte. Idle, optionale Timer-Task, weitere Objekte, Speicherzuschläge und Ausrichtung fehlen. Die tatsächliche TCB-Größe ist versions- und konfigurationsabhängig. Der Handlerstack und übrige Daten müssen ebenfalls in die Gesamtspeicherbilanz. Bereits im Heap enthaltene Taskblöcke werden nicht zusätzlich außerhalb dieses Heaps gezählt. $128 \cdot 4 = 512$ Byte; CMSIS `stack_size=512` ist dagegen bereits Byte.
2. Yield fordert Auswahl an, macht HeavyMath aber nicht Blocked. Die höchste bereite Priorität bleibt HeavyMath. Heartbeat verhungert: Starvation. Es gibt keine gemeinsame Ressource und keine Warteabhängigkeit zu einem niedrigen Ressourcenbesitzer; daher keine Priority Inversion.
3. Logische Priorität 3 ist dringlicher als 5 und darf in dieser Konfiguration keine Kernel-API verwenden. Auch ein erlaubter IRQ darf nicht mit `osThreadFlagsWait` blockieren; er darf das passende Bit mit `osThreadFlagsSet` setzen. Ein Bit bleibt bei zweimaligem Setzen vor dem Wait nur gesetzt und zählt nicht verlustfrei zwei Ereignisse.

Teil B: Programmieren und Beobachten

Aufgabe 5: Projekt mit CubeMX und VS Code vorbereiten

Werkzeuge: STM32CubeMX, STM32CubeH7-Paket, VS Code mit offizieller STM32 VS Code Extension und deren Build-/Debugwerkzeugen, USB-Verbindung zum ST-LINK des Boards. ST-LINK ist der integrierte Programmier- und Debugadapter. Ein erfolgreicher Dokumentbuild ist kein Firmwarebuild.

1. Notieren Sie die Versionen von CubeMX, CubeH7, Extension, Compiler, FreeRTOS-Kernel und CMSIS-Adapter. Wählen Sie das Board **NUCLEO-H755ZI-Q**, MCU **STM32H755ZIT6**. Erzeugen Sie mit dem **CMake-Generator** die Projektteile für Cortex-M7 und Cortex-M4. CMake beschreibt Übersetzen und Verknüpfen. Öffnen Sie das erzeugte Projekt in VS Code und wählen Sie die vorgesehenen STM32-Build-/Debugfunktionen.

   Prüfen Sie vor weiteren Änderungen, dass der installierte Generator die Kombination **Dual-Core, CMake und FreeRTOS auf CM4** unterstützt und tatsächlich zwei passende Ziele erzeugt. Menübezeichnungen und Dateistruktur sind versionsabhängig. Fehlt diese Unterstützung, dokumentieren Sie die konkrete Diagnose und verwenden Sie mit der Lehrperson einen passenden aktuellen Werkzeugstand oder ein bereitgestelltes Dual-Core-CMake-Projekt. Ein nur für CM7 erzeugtes Projekt erfüllt die Aufgabe nicht. Wechseln Sie nicht still zu CubeIDE.

2. Übernehmen bzw. prüfen Sie die Bootfolge aus Vorlesung 1. CM7 enthält die System-/Startfunktion; CM4 die Anwendung. Laden Sie beide passenden Images, prüfen Sie beide Resets und das Erreichen der CM4-Anwendung. M7-400-MHz und M4-200-MHz sind Lehrvorgaben, keine automatisch erzeugten Taktraten. Wenn Sie vorhandene Firmware aus Vorlesung 1/2 verwenden, sichern Sie diese vor Flashänderungen.

3. Konfigurieren Sie FreeRTOS **nur auf CM4**, Schnittstelle **CMSIS_V2**, Präemption aktiv, 32-Bit-Kernelticks, Labor-Tickfrequenz 1000 Hz. Prüfen Sie im erzeugten `FreeRTOSConfig.h` die Werte und im Adapter `cmsis_os2.c` die Prioritätsabbildung. `configMAX_PRIORITIES` muss die gewünschten CMSIS-Prioritäten unterstützen. Wählen Sie die Heapgröße anhand der tatsächlich erzeugten Objekte; 16 KiB kann ein Startwert sein, ist kein bestätigtes Minimum.

4. Laborvariante: SysTick für den Kernel, einen freien **TIM6** für die HAL-Zeitbasis. Prüfen Sie Tickfrequenzen und Handlerketten. Ein gemeinsam korrekt integrierter SysTick wäre möglich, wird in diesem Versuch jedoch nicht verwendet.

5. Konfigurieren Sie **LD2/PE1** und **LD1/PB0** als Ausgänge für CM4. Verifizieren Sie Boardrevision und Belegung gegen UM2408 bzw. den Schaltplan. Konfigurieren Sie Benutzer-Taster **PC13** mit passender Flanke und EXTI-Leitung für CM4. EXTI ist der External Interrupt/Event Controller. Prüfen Sie den elektrischen Ruhepegel, statt eine Flankenrichtung zu raten.

6. Der Taster-IRQ **EXTI15_10** muss für Kernelaufrufe erlaubt sein und zu der in Aufgabe 4 erläuterten Bibliotheksschwelle passen. In diesem Labor darf keine weitere EXTI-Leitung diesen IRQ benutzen. Lassen Sie die NVIC-Freigabe bei der Generierung aus. NVIC bedeutet Nested Vectored Interrupt Controller. Falls CubeMX sie zwingend erzeugt, sperren Sie den IRQ unmittelbar nach der GPIO-Initialisierung in einem erhaltenen USER-CODE-Block, noch bevor ein gültiges Threadhandle vorausgesetzt wird. Er darf vor dem Kernelstart keinen Flagaufruf auslösen.

7. Bauen Sie beide **noch unveränderten** Ziele in VS Code. Kontrollieren Sie Image, Kernzuordnung, Linkerskript und Map-Datei. Dokumentieren Sie die tatsächlichen Erfolgsmeldungen oder Fehler. Eine Map-Datei ist der Speicherbericht des Linkers. Erst nach funktionierendem Dual-Core-Start integrieren Sie die Threads.

Abgabe: Versionstabelle, `.ioc`, Build-/Debugkonfiguration sowie Nachweis der beiden erzeugten und geladenen Ziele. Wenn Hardware oder Werkzeugunterstützung fehlt, bleibt Teil A vollständig bearbeitbar; dokumentieren Sie Teil B als nicht durchgeführt.

Erwartete Kontrolle: Zwei korrekt zugeordnete Images mit konsistenter Bootfolge, Kernel ausschließlich auf CM4. Der CM4-Kontext erreicht die Kernelinitialisierung. SysTick bedient die konfigurierte Kernelzeit, die erzeugte TIM6-Handlerkette die HAL-Zeitbasis. Ungeprüfte gemeinsame Handler und ein mitaktivierter Kernel auf CM7 sind keine Lösungen. Eine bestimmte hier erfundene Extension-Menüfolge oder ein angeblich getestetes Firmwareprojekt wird nicht vorausgesetzt.

Aufgabe 6: Drei Threads integrieren

Legen Sie in CubeMX diese drei CMSIS-RTOS2-Threads an. Benennen Sie einen eventuell erzeugten Default-Thread entsprechend um oder entfernen Sie ihn; eine zusätzliche hochpriore Endlosschleife würde den Versuch verändern.

Rolle	CMSIS-Priorität	Stackattribut in Byte
Ereignis-Task	`osPriorityHigh`	1024
Heartbeat	`osPriorityNormal`	512
HeavyMath	`osPriorityBelowNormal`	1024

Die Größen sind Ausgangswerte, keine nachgewiesenen sicheren Minima. Prüfen Sie die Einheit der CubeMX-Felder gegen `.stack_size` im generierten Code.

Anwendungsbaustein: Kopieren Sie `code/lab03/lab03_rtos.c` und `lab03_rtos.h` aus dem Kursrepository in einen eigenen CM4-Anwendungsordner. Fügen Sie Quelle und Include-Verzeichnis dem **CM4-CMake-Ziel** hinzu, vorzugsweise über dessen vorhandene Benutzer-Erweiterungsstelle. Der Baustein ist nicht für CM7 vorgesehen. Er benötigt die erzeugten Header `main.h` und `cmsis_os2.h`, GPIO-Initialisierung und Kernelkonfiguration. Kontrollieren Sie, dass eine erneute CubeMX-Generierung diese Einbindung und Ihre USER-CODE-Bereiche erhält.

1. Binden Sie `lab03_rtos.h` im erhaltenen Include-Bereich der erzeugten Threaddatei und der Datei mit dem HAL-Callback ein.
2. Ersetzen Sie den generierten Schleifenkörper jeder Eintrittsfunktion durch **einen** Aufruf des passenden Bausteins. Erzeugen Sie keine zweite Definition der Threadfunktion. Beispiel für die vorhandene Heartbeat-Funktion:

```
/* Innerhalb der generierten Heartbeat-Eintrittsfunktion */
Lab03_Heartbeat(argument); /* tatsächlichen Argumentnamen verwenden */
```

Analog rufen HeavyMath und Ereignis-Task `Lab03_HeavyMath(argument)` bzw. `Lab03_Event(argument)` auf. Die Funktionen kehren nicht zurück. Code gehört in Funktions-USER-CODE-Bereiche, nicht in `Header_...`-Kommentare.

3. Prüfen Sie nach dem vorhandenen `osKernelInitialize` alle drei von `osThreadNew` erzeugten Handles auf `NULL`. Nach erfolgreicher Anlage, **vor** dem einzigen vorhandenen `osKernelStart`, rufen Sie auf:

```
Lab03_BindEvent(eventTaskHandle); /* Ihren generierten Handle-Namen einsetzen */
```

Der Kernel muss zu diesem Zeitpunkt im Zustand `osKernelReady` sein. `osKernelStart` kehrt bei Erfolg in der üblichen Integration nicht zurück; eine Rückkehr gehört in die Fehlerbehandlung.

4. Verwenden Sie den **bestehenden** `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`. Fügen Sie darin `Lab03_ButtonIRQ(GPIO_Pin);` ein. Falls noch kein eigener Callback existiert, definieren Sie einmalig die von der HAL vorgesehene Funktion im erhaltenen Benutzerbereich. Definieren Sie keinen zusätzlichen `EXTI15_10_IRQHandler`: Die generierte Interrupt-/HAL-Kette muss weiter genau einmal zuständig bleiben.
5. Lesen Sie den Baustein. Er setzt beim Start der Ereignis-Task alte PC13-Pending-Markierungen zurück und gibt danach den IRQ frei. Diese Startpolitik verwirft Flanken vor Betriebsbeginn. Danach werden angenommene Thread-Flags durch `osThreadFlagsWait` automatisch gelöscht, ohne zusätzliches blindes Löschen.
6. Bauen und laden Sie wieder beide benötigten Images. Die Anwendungsquelle ist ein Lehrbaustein, kein schon auf Ihrem Board getesteter Komplettstand. Prüfen Sie mindestens eine erneute Generierung mit anschließendem Build auf Erhalt der Integration.

Beantworten Sie zusätzlich: Warum darf die ISR (Interrupt Service Routine) hier `osThreadFlagsSet`, aber nicht `osThreadFlagsWait` aufrufen? Warum ist `osOK` der falsche Erfolgstest für Flagfunktionen?

Begründung: `osThreadFlagsSet` ist für zugelassenen Interruptkontext vorgesehen und wartet nicht. Der FreeRTOS-Port muss die IRQ-Priorität trotzdem erlauben. `osThreadFlagsWait` blockiert den aufrufenden Thread und gehört nicht in eine ISR. Flagfunktionen liefern Flagbits

oder einen Wert mit reserviertem Fehlerbit. Daher wird `(flags & osFlagsError) != 0u` geprüft; `osOK` ist ein Statuscode anderer Aufrufe.

Bausteinverhalten: `Lab03_Heartbeat` konvertiert 500 ms mit 64-Bit-Zwischenprodukt und Aufrundung in eine positive Tickzahl. `LabWaitNext` erhöht den vorherigen Solltermin. Bei abgelaufenem Ziel zählt es `lab03_target_misses`, richtet auf jetzt plus Periode neu aus und wartet. Ein zwischen Prüfung und Adapteraufruf verpasstes Ziel wird einmal ebenso behandelt; weitere Fehler führen zu `Error_Handler`. Der Zähler bezeichnet Wartezielverfehlungen, keine vollständige Deadlineanalyse.

`Lab03_HeavyMath` bearbeitet eine eigene vorzeichenlose `volatile`-Variable dauerhaft und bleibt Ready. `volatile` erhält Speicherzugriffe, bietet aber keine gemeinsame Datensynchronisation. `Lab03_Event` wartet auf Bit null und schaltet LD1/PB0 um. LD2/PE1 bleibt ausschließlich Heartbeat zugeordnet. Die Ergebnisse `lab03_event_responses` zählen angenommene Hinweise, nicht verlustfrei einzelne physische Tastendrücke.

Aufgabe 7: Normalbetrieb und Starvation vergleichen

Normalbetrieb

- Heartbeat schaltet LD2 ungefähr alle 500 ms um, obwohl HeavyMath nicht wartet. Ein ganzer LED-Zyklus dauert ungefähr eine Sekunde.
- Tasterhinweise schalten LD1 um. Ein mechanischer Taster kann prellen. Das Flag beschreibt **mindestens einen ausstehenden Reaktionswunsch**, keinen garantierten Zähler entprellter Tastendrücke.
- Kontrollieren Sie die Debugvariablen `lab03_accumulator`, `lab03_irq_errors`, `lab03_target_misses` und `lab03_event_responses`. Ihre Werte sind Beobachtungen des tatsächlich durchgeführten Laufs. Halten im Debugger verändert das Zeitverhalten; nutzen Sie die Variablen nicht als ungestörte Zeitmessung.
- Begründen Sie die erwarteten Zustände aller drei Threads zwischen zwei Heartbeat-Aktionen.

Geänderte Priorität

1. Ändern Sie nur HeavyMath auf `osPriorityAboveNormal`, also über Heartbeat und unter Ereignis-Task. Vermeiden Sie gleiche Prioritäten. Erzeugen, bauen und starten Sie neu.
2. Dokumentieren Sie LD2 und LD1. Die Heartbeat-LED darf in ihrem letzten Zustand stehen bleiben oder schon vor ihrer ersten Aktion gar nicht beginnen; beides passt zu fehlender Zuteilung.
3. Erklären Sie, warum `osThreadYield()` in HeavyMath Heartbeat nicht rettet. Was passiert dagegen bei echtem Betriebssystem-Warten in HeavyMath?
4. Stellen Sie `osPriorityBelowNormal` wieder her und prüfen Sie den Normalbetrieb erneut.

Abgabe: Gegenüberstellung beider Konfigurationen, tatsächliche Beobachtungen und begründete Diagnose. Eine stehengebliebene LED allein ist noch kein Beweis für die Ursache. Prüfen Sie Handle, Kernelstart, Boot, GPIO, Ready-Zustand und tatsächliche Priorität.

Im Normalbetrieb sind Heartbeat und Ereignis-Task zwischen Aktionen Blocked, HeavyMath Running. Bei Freigabe einer höheren Aufgabe ist HeavyMath Ready, während diese läuft. Der Normalversuch demonstriert präemptive Zuteilung; sichtbar ruhiges Blinken beweist keine harte Frist und keinen Jitter von null.

Mit HeavyMath über Heartbeat ist Heartbeat Ready ohne Zuteilung: Starvation. Die Ereignis-Task bleibt höher und kann reagieren. Yield nimmt HeavyMath nicht aus der Ready-Menge.

Echtes Warten macht HeavyMath dagegen Blocked und erlaubt Heartbeat die Auswahl, wenn keine noch höhere Aufgabe bereit ist. Dieses Verhalten ist keine Priority Inversion, weil keine Ressource gehalten wird.

Zusatzaufgabe: Messen statt nur Blinken beobachten

Falls ein Oszilloskop oder Logikanalysator verfügbar ist, legen Sie einen **separaten freien GPIO-Marker** um die Heartbeat-Arbeit. Pin und Boardanschluss müssen anhand des Schaltplans gewählt werden; benutzen Sie dafür nicht die von einem anderen Thread geschriebene LED.

- Unterscheiden Sie Marker-Pulsbreite $w_k = f_k - s_k$ und Startabstand $P_k = s_{k+1} - s_k$. s_k ist die Start-, f_k die Endmarke. Eine Pulsbreite kann Präemption oder Interruptarbeit einschließen.
- Notieren Sie Werkzeugstand, Taktraten, Tickrate, Messpin, Optimierung, Last und Geräteauflösung.
- Erläutern Sie, warum eine gemessene maximale Pulsbreite in dieser endlichen Testreihe noch keine allgemein nachgewiesene größte Ausführungszeit ist.
- Falls keine Messung stattfand, tragen Sie keine Modellzahlen als Messergebnis ein.

Abgabeübersicht

Teil A: begründete Lösungen zu Aufgaben 1–4 (32 Punkte). Teil B: Projektkonfiguration und Versionsliste, Quellen und Build-/Debugeinstellungen, Integrations- und Regenerierungsprüfung, Beobachtungsvergleich mit wiederhergestelltem Normalbetrieb. Trennen Sie bestanden, fehlgeschlagen und nicht durchgeführt.

Quellen: Vorlesung 3; Arm CMSIS-RTOS2-Dokumentation zu Wait, Threads und Thread Flags; FreeRTOS-Dokumentation zum Cortex-M-Port; ST UM2408, RM0399 und AN5361; Dokumentation der installierten STM32 VS Code Extension. Ihre Projektversionen haben Vorrang vor Annahmen über den aktuellen öffentlichen Entwicklungszweig.